Welcome to CS300

CS300: Programming II

Cole Nelson

Before We Begin

- View your Canvas announcements, important information comes from there! :)
- Starting this week, readings come **before** lecture.
  - Missed it? You still have time this week...
  - *Very important* for future lectures! We spend time in hands-on examples.
- Not invited to TopHat? Use code **800239**

Learning Objectives

1. Be able to **identify** what work is due, and when, across the CS300 weekly study cycle.
2. Be able to **explain** why an array's `length` field doesn't tell you how many elements are in use.
3. Be able to **classify** an array as perfect-size or oversize from a code snippet.

Your Course Staff

- There's a lot of us!
  - **3 instructors** (Hobbes, Cole [that's me!], Ben)
  - **9 graduate TAs**
  - **3 undergraduate peer mentors**.
- One email reaches all of us: **cs300@cs.wisc.edu**.
  - Even better: **use Piazza.**

Who are you?

1 minute to share...

1. Your name.
2. Your *intended* major.
3. Something fun you did over the summer.

What is CS300?

Programming II – your *second* programming course.

- From small snippets to **multi-class programs**.
- OO-design: interfaces, generics, exceptions.
- Foundational **data structures**: lists, stacks, trees.
- Reasoning about **efficiency**, not just correctness.

Take self-assessment to decide if this class is for you!

Two Critical Skills

- **Writing fluency** – constructing correct, efficient, well-organized code.
- **Reading fluency** – understanding and reasoning about code; possibly that you *didn't* write.

Our hypothesis is that being able to *write* computer programs will better help you *read* them for critical mistakes. See **AI statement**...

AI Statement

You build the programs, not an agent. Ask it to help, not to solve; to guide, not to direct.

Use `log.txt` and 'citations' to cite your usage.

```
1. AI is the best tool ever invented for learning.
2. AI is the best tool ever invented for not learning.
  - Zvi Mowshowitz
```

AI Statement

This is a **foundational course** teaching **foundational skills**. Outsourcing your learning this early in your career does *not* set you up for success!

Concepts practiced on homeworks will be tested on exams; it's in your best interest to put in the work!

How Your Grade Breaks Down

Category	Weight
Exams	50%
Programming assignments	25%
Weekly activities	25%

How This Course Works

See the syllabus!

TopHat Q1

When are zyBooks **participation activities** due?

Feel free to discuss with a neighbor!

Built-in Flexibility: the 80% Rule

For weekly formative work (zyBooks, Top Hat, Canvas activities, program quizzes, and the *formative* half of programs):

- Finish **≥ 80% on time** → **100%**.
- Below 80% → scaled proportionally.

Life happens — no documentation needed for the occasional miss.

Deadlines & When Life Happens

- **Deadlines are final**: `11:59:00 PM` is on time, `11:59:01` is not. Submit early, even if only for partial completion credit.
- The 80% rule already absorbs short illness, tech hiccups, and busy weeks.

Where to Get Help

- **Drop-in hours** with TAs and peer mentors – schedule pending on Canvas.
- **Piazza**, public or private posts.
- Course staff email: **cs300@cs.wisc.edu**.
- No computer? UW **laptop lending** (library, DoIT, and the CS department).

You're surrounded by support – use it early and often.

Arrays, Revisited

The rest of today is a preview of next lecture.

Arrays, Quickly

- An array has a **fixed length**, chosen when it's created.
- `arr.length` is a **field**, not a method — no parentheses.
- Indexing is **zero-based**: the valid range is `[0, arr.length - 1]`.
- Unassigned slots hold a **default**.

Example 1: How Full Is This Array?

We reserve room for 5 students, but only 3 have signed up.

```
String[] roster = new String[5];
roster[0] = "Amara";
roster[1] = "Bao";
roster[2] = "Mateo";
```

What will `roster.length` print — 3 or 5?

→ Step through it in Java Visualizer.

Example 1: What We Saw

```
roster.length = 5
roster[3] = null
roster[4] = null
```

`length` is the **capacity we reserved (5)** — not the **3 students** actually signed up. An array **can't count itself**.

Tophat Q2

A quick check on `length` vs. how full an array is.

→ Open Top Hat to vote.

Two Ways to Store "a List in an Array"

Strategy	What `length` means	Track a `size`?
Perfect-size	exactly the live count	No
Oversize	reserved capacity	Yes

Choose **oversize** when the collection **grows or shrinks** at runtime.

Oversize = Array plus a `size` Counter

- The array length gives you **capacity**; a separate `size` variable gives you the **count**.
- Live elements sit in `[0, size - 1]`; the rest is reserved capacity.
- One rule holds always: **`size <= arr.length`**.

Example 2: Spot the Type

Perfect-size or oversize? Discuss each with a neighbor.

```java
// (a)
String[] team = {"Red", "Blue", "Green"};

// (b)
String[] queue = new String[8];
int size = 0;
```

Example 2: Debrief

- **(a)** Perfect-size — no `size` counter, so all three slots *are* the data.
- **(b)** Oversize — capacity is 8, but `size = 0` says "nothing live yet."

Tophat Q3

A quick check on classifying perfect-size vs. oversize.

→ Open Top Hat to vote.

Questions?